

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I Ajay Velmail E (192311385) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ E Ajay Velmail _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or

	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

ARCHITECTURE DESIGN ASSIGNMENT

Project: Cold-Chain Logistics Integrity Platform

1. ANALYZE SYSTEM REQUIREMENTS

1.1 System Overview

The **Cold-Chain Logistics Integrity Platform** is a distributed software system designed to continuously monitor temperature-sensitive food and pharmaceutical shipments during transportation and storage.

The system collects real-time telemetry from IoT sensors installed in refrigerated trucks, containers, warehouses, and storage facilities. The telemetry includes:

- Temperature
- Humidity
- GPS location
- Door status
- Refrigeration-unit health
- Timestamp
- Shipment information

The collected information is processed continuously to detect environmental excursions, identify potential shipment failures, notify relevant stakeholders, and generate compliance documentation.

The architecture must therefore support **large-scale IoT connectivity, real-time data processing, high availability, security, fault tolerance, scalability, and historical analytics.**

1.2 System Objectives

The major objectives of the system are:

1. Continuously collect data from IoT sensors.
2. Monitor shipment conditions in real time.
3. Detect environmental deviations automatically.
4. Apply product-specific temperature and humidity thresholds.
5. Detect refrigeration-unit problems.
6. Track shipment locations using GPS.
7. Predict potential shipment failures.
8. Generate immediate alerts.
9. Provide historical analytics.
10. Generate automated compliance reports.
11. Maintain complete audit trails.
12. Support thousands of simultaneously connected vehicles.
13. Provide secure data transmission.
14. Maintain high system availability and fault tolerance.

1.3 Functional Requirements

The architecture must support the following functional requirements.

FR1 – User Authentication

The system shall allow authorized users to securely log in to the platform.

FR2 – User and Role Management

The system shall allow administrators to create users and assign appropriate roles and permissions.

FR3 – IoT Device Registration

The system shall allow authorized administrators to register and manage IoT sensors.

FR4 – Real-Time Telemetry Collection

The system shall continuously receive telemetry from connected IoT devices.

FR5 – Environmental Monitoring

The system shall monitor:

- Temperature
- Humidity
- GPS location
- Door status
- Refrigeration-unit health

FR6 – Shipment Management

The system shall allow shipments to be created and associated with vehicles, containers, products, and sensors.

FR7 – Threshold Management

The system shall allow product-specific acceptable environmental ranges to be configured.

FR8 – Excursion Detection

The system shall automatically detect environmental conditions that exceed configured thresholds.

FR9 – Alert Generation

The system shall generate alerts when critical environmental events are detected.

FR10 – Notification

The system shall notify relevant stakeholders about critical incidents.

FR11 – GPS Tracking

The system shall display the current or latest known location of shipments.

FR12 – Predictive Failure Detection

The system shall analyze telemetry and identify potential shipment or refrigeration failures.

FR13 – Historical Analytics

The system shall store and analyze historical environmental and shipment data.

FR14 – Compliance Reporting

The system shall generate automated compliance documentation.

FR15 – Audit Trail

The system shall maintain records of important events, alerts, and system activities.

1.4 Non-Functional Requirements

1. Scalability

The system must support **thousands of simultaneously connected vehicles and IoT sensors** without significant degradation in performance.

2. Performance

Telemetry must be processed with low latency so that environmental excursions can be detected quickly.

3. Availability

The platform should remain available continuously because shipment monitoring is required throughout transportation and storage.

4. Reliability

The system must reliably collect, process, and store telemetry without losing important environmental information.

5. Security

Sensor data, shipment information, user accounts, and compliance records must be protected from unauthorized access and modification.

6. Maintainability

Individual components should be independently maintainable and replaceable without affecting the entire system.

7. Fault Tolerance

The failure of a single service, sensor, or communication component should not bring down the entire platform.

8. Usability

Dashboards should clearly display shipment conditions, alerts, locations, and risk information.

9. Interoperability

The platform should be capable of communicating with different IoT devices and external logistics systems.

10. Data Integrity

Telemetry and compliance information must remain accurate, consistent, and traceable.

1.5 Quality Attributes Influencing Architecture

The most important quality attributes are:

Quality Attribute	Architectural Requirement
Scalability	Handle thousands of IoT devices and high-volume telemetry
Security	Secure devices, APIs, users, and data
Performance	Process telemetry and alerts with low latency
Reliability	Prevent data loss and ensure consistent processing
Availability	Provide continuous monitoring
Maintainability	Use modular and independently deployable components
Fault Tolerance	Continue operating despite individual component failures
Usability	Provide clear dashboards and actionable alerts
Interoperability	Support multiple devices and external systems

2. SELECT A SUITABLE SOFTWARE ARCHITECTURE

2.1 Selected Architecture

Hybrid Microservices + Event-Driven Architecture

The most suitable architecture for the Cold-Chain Logistics Integrity Platform is a:

Hybrid Microservices and Event-Driven Architecture

This combines the advantages of **microservices** for modular business functionality with an **event-driven architecture** for processing large volumes of real-time IoT telemetry.

The assessment specifically allows architectural styles such as Microservices, Event-Driven, Service-Oriented, Serverless, and Hybrid Architecture.

2.2 Why This Architecture is Suitable

1. Large-Scale IoT Data

Thousands of sensors can continuously produce telemetry.

An event-driven approach allows incoming sensor events to be distributed to multiple consumers without tightly coupling the components.

2. Independent Scaling

Microservices can be scaled independently.

For example:

- Telemetry service can scale when sensor traffic increases.
- Notification service can scale during large numbers of alerts.
- Analytics service can scale when historical analysis is requested.

3. Real-Time Processing

Environmental events can be processed immediately.

For example:

Sensor → Event Broker → Monitoring Service → Threshold Engine → Alert Service

This supports fast detection of temperature excursions.

4. Fault Isolation

If one service fails, other services can continue operating.

For example, failure of the reporting service should not stop telemetry collection.

5. Maintainability

Each major business function is separated into its own service.

Developers can modify the alert service without changing the shipment-management service.

6. Future Expansion

New services can be added without redesigning the entire system.

Examples include:

- AI prediction service
- Route-risk service
- Mobile service
- Advanced analytics
- Digital twin service

2.3 Major Architectural Layers

The proposed architecture can be divided into the following layers:

Layer 1 – IoT Device Layer

Contains:

- Temperature sensors
- Humidity sensors
- GPS devices
- Door sensors
- Refrigeration sensors

Layer 2 – Edge/Gateway Layer

Responsible for:

- Collecting sensor data
- Validating basic telemetry
- Temporary buffering
- Secure communication
- Forwarding telemetry to the cloud

Layer 3 – Communication/Event Layer

Contains:

- IoT gateway
- API gateway
- Message broker/event bus

This layer distributes telemetry events to required services.

Layer 4 – Microservices Layer

Contains independent business services.

Layer 5 – Data Layer

Contains databases for:

- Telemetry
- Shipments
- Users
- Devices
- Events

- Alerts
- Audit records

Layer 6 – Analytics and AI Layer

Responsible for:

- Historical analysis
- Anomaly detection
- Predictive failure detection
- Risk scoring

Layer 7 – Presentation Layer

Contains:

- Web dashboard
- Mobile application
- Administrative interface
- Compliance interface

Layer 8 – External Integration Layer

Provides integration with:

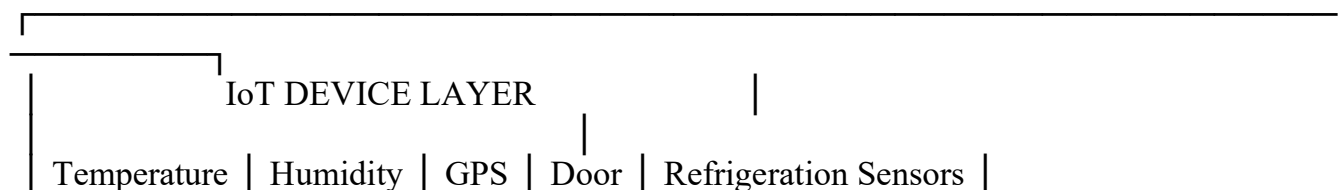
- Logistics systems
- Regulatory systems
- Notification services
- External APIs

3. SOFTWARE ARCHITECTURE DIAGRAM

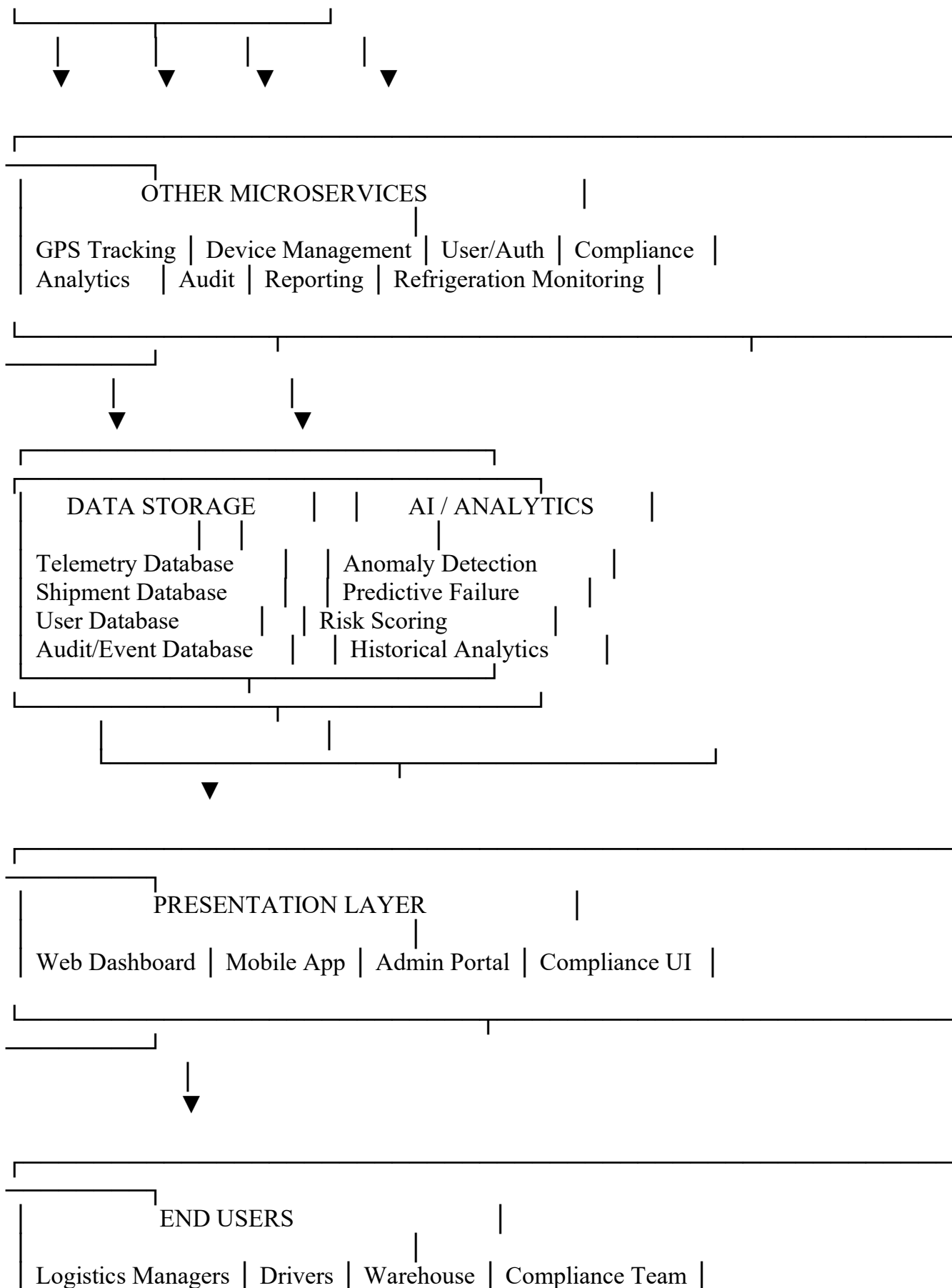
The following diagram represents the proposed architecture.

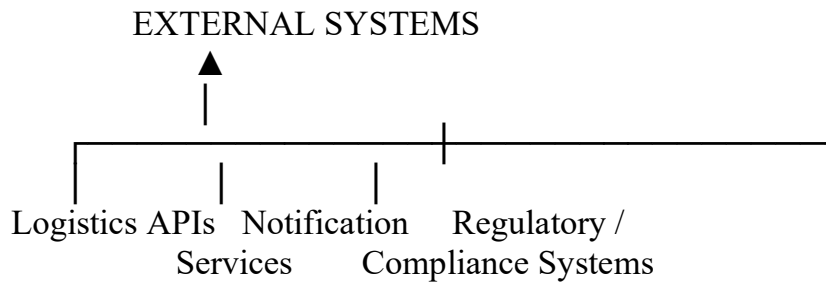
COLD-CHAIN LOGISTICS INTEGRITY PLATFORM

=====









3.1 Communication Mechanisms

Different communication mechanisms can be used depending on the interaction.

Communication	Purpose
MQTT	Lightweight IoT telemetry communication
HTTPS/REST API	Application and external system communication
Message Broker	Asynchronous event distribution
WebSocket	Real-time dashboard updates
Database APIs	Data access between services
Notification APIs	SMS/email/push notifications

4. EXPLAIN COMPONENTS AND THEIR INTERACTIONS

4.1 IoT Sensors

Sensors continuously collect environmental and operational information.

Responsibilities

- Measure temperature
- Measure humidity
- Capture GPS coordinates
- Detect door openings
- Monitor refrigeration-unit health
- Send telemetry

4.2 Edge/IoT Gateway

The gateway acts as an intermediate layer between sensors and the cloud platform.

Responsibilities

- Authenticate devices
- Validate sensor information
- Temporarily store data during connectivity loss
- Forward telemetry
- Reduce unnecessary network traffic

4.3 API Gateway

The API Gateway provides a controlled entry point into backend services.

Responsibilities

- Request routing
- Authentication
- Authorization
- Rate limiting
- Load balancing
- API management

4.4 Message Broker

The message broker is a key component of the event-driven architecture.

It receives telemetry events and distributes them to appropriate services.

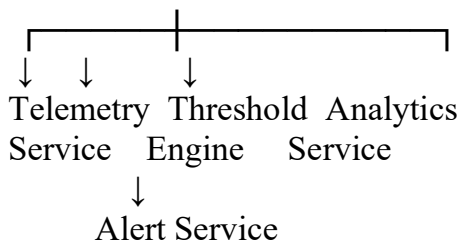
For example:

Temperature Event



Message Broker





This reduces direct dependencies between services.

4.5 Telemetry Service

The telemetry service processes incoming sensor data.

Responsibilities

- Receive telemetry
- Validate data
- Timestamp data
- Normalize data
- Store telemetry
- Publish relevant events

4.6 Shipment Service

This service manages shipment-related information.

Responsibilities

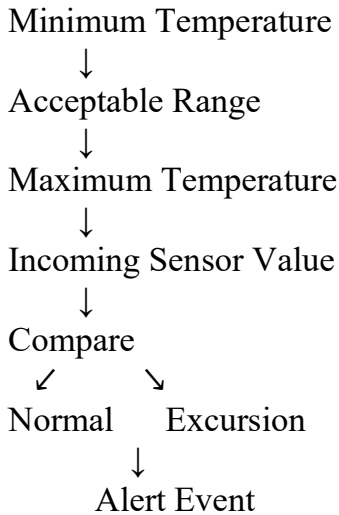
- Create shipments
- Assign products
- Assign vehicles
- Associate sensors
- Track shipment status

4.7 Threshold Engine

The threshold engine determines whether sensor values are within acceptable limits.

Example:

Product: Temperature-Sensitive Medicine



4.8 Alert and Notification Service

This service handles incidents detected by the monitoring system.

Responsibilities

- Generate alerts
- Classify severity
- Notify stakeholders
- Escalate unresolved incidents
- Maintain alert history

4.9 GPS Tracking Service

This service processes location information.

Responsibilities

- Track vehicles
- Store locations

- Display shipment routes
- Identify delays
- Associate incidents with locations

4.10 Device Management Service

This service manages IoT devices.

Responsibilities

- Device registration
- Device configuration
- Device status
- Connectivity monitoring
- Device health monitoring

4.11 Analytics Service

The analytics service processes historical data.

Responsibilities

- Generate trends
- Analyze shipment performance
- Identify recurring excursions
- Analyze refrigeration performance
- Generate operational insights

4.12 AI/Predictive Analytics Service

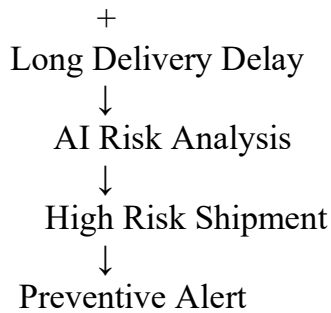
This service uses historical and real-time telemetry to identify potential failures.

Example

Temperature Increasing

+

Refrigeration Health Decreasing



4.13 Compliance Service

This service automatically generates regulatory documentation.

Responsibilities

- Generate compliance reports
- Record shipment history
- Record environmental excursions
- Provide audit evidence
- Export required reports

4.14 Audit Service

The audit service records important system events.

Examples:

- User login
- Configuration changes
- Threshold changes
- Alert generation
- Alert acknowledgement
- Report generation

4.15 Data Storage

Different types of data can be stored using appropriate databases.

Telemetry Database

Stores high-volume sensor readings.

Transactional Database

Stores:

- Users
- Shipments
- Products
- Vehicles
- Devices

Event/Audit Database

Stores:

- Alerts
- Excursions
- Audit records
- System events

This separation improves scalability and maintainability.

4.16 Presentation Layer

The presentation layer provides interfaces for different users.

Logistics Dashboard

Provides:

- Fleet overview
- Shipment status
- Environmental conditions
- Alerts
- GPS location

Compliance Dashboard

Provides:

- Shipment reports
- Environmental history
- Excursion records
- Audit information

Administrator Portal

Provides:

- User management
- Device management
- Threshold configuration
- System configuration

4.17 Overall System Workflow

The complete workflow is:

Step 1 – Data Collection

IoT sensors collect temperature, humidity, GPS, door, and refrigeration data.

↓

Step 2 – Edge Processing

The gateway validates and temporarily buffers the information.

↓

Step 3 – Secure Transmission

Data is securely transmitted to the cloud platform.

↓

Step 4 – Event Ingestion

The API Gateway forwards telemetry to the message broker.

↓

Step 5 – Event Distribution

The broker distributes telemetry events to relevant microservices.



Step 6 – Processing

Telemetry Service validates and stores the information.



Step 7 – Rule Evaluation

Threshold Engine compares sensor values against product-specific limits.



Step 8 – Excursion Detection

If an environmental condition is outside the acceptable range, an excursion event is generated.



Step 9 – Alert Generation

Alert Service creates an appropriate alert and notifies stakeholders.



Step 10 – Analytics

Analytics and AI services analyze current and historical information to identify risks.



Step 11 – Dashboard

Users view real-time shipment conditions and alerts.



Step 12 – Compliance

Compliance Service generates reports and maintains audit information.

5. QUALITY ATTRIBUTE ANALYSIS

The architecture must address scalability, security, performance, reliability, maintainability, and availability, which are explicitly required by the assignment.

5.1 Scalability

Requirement

The platform must support thousands of simultaneously connected vehicles and sensors.

Architectural Solution

The microservices and event-driven architecture allows individual services to scale independently.

For example:

Low Sensor Traffic



Few Telemetry Instances

High Sensor Traffic



More Telemetry Instances



Same Platform

The message broker also helps distribute high-volume telemetry across multiple processing instances.

Result

The system can grow from a small fleet to a large logistics network without completely redesigning the architecture.

5.2 Security

Security is required at multiple levels.

Device Security

- Device authentication
- Unique device identities
- Secure communication

Network Security

- Encrypted communication
- API authentication
- Network access controls

Application Security

- User authentication
- Role-based access control
- Input validation
- Secure APIs

Data Security

- Encryption
- Access control
- Audit logging
- Data integrity mechanisms

Result

Unauthorized users and devices are prevented from accessing or manipulating sensitive shipment information.

5.3 Performance

Requirement

Environmental excursions must be detected quickly.

Architectural Solution

The event-driven architecture allows telemetry to be processed asynchronously instead of forcing every service to communicate synchronously.

The system can also use:

- Message queues
- Caching
- Load balancing
- Independent service scaling
- Efficient data storage

Result

Real-time monitoring and alert generation can be achieved with low processing latency.

5.4 Reliability

Reliability ensures that sensor data and monitoring functions operate correctly.

Architectural Mechanisms

- Persistent telemetry storage
- Message durability
- Retry mechanisms
- Error handling
- Data validation
- Redundant services
- Health monitoring

If one processing instance fails, another instance can continue processing events.

5.5 Maintainability

Microservices improve maintainability by separating business responsibilities.

For example:

Alert Requirement Changes



Modify Alert Service



Other Services Remain Unchanged

This reduces the risk of introducing defects into unrelated parts of the system.

5.6 Availability

The platform must provide continuous monitoring.

Techniques

- Multiple service instances
- Load balancing
- Automatic service recovery
- Redundant infrastructure
- Database replication
- Message persistence
- Health checks

The failure of one component should not cause complete system downtime.

6. JUSTIFY ARCHITECTURAL DECISIONS

6.1 Why Microservices?

Microservices are selected because the platform contains several independent business functions.

Examples:

- Telemetry
- Shipment management
- Alerts
- GPS
- Device management
- Compliance
- Analytics

Each can be independently developed, tested, deployed, and scaled.

6.2 Why Event-Driven Architecture?

IoT devices continuously generate events.

An event-driven architecture is therefore suitable because it allows:

- Asynchronous processing
- High-throughput telemetry handling
- Loose coupling
- Real-time event processing
- Independent consumers
- Better fault isolation

6.3 Why a Hybrid Architecture?

Using only microservices would not fully address the high-volume event-streaming nature of IoT telemetry.

Using only an event-driven architecture could make conventional business operations more difficult to organize.

Therefore, combining both approaches provides a balanced solution:

Microservices → Business functionality

Event-driven → Real-time telemetry processing

6.4 Trade-Offs

Advantages

High Scalability

Individual services can scale independently.

Better Fault Isolation

A failure in one service does not necessarily stop the entire platform.

Real-Time Processing

Events can be processed as soon as telemetry arrives.

Maintainability

Services have clearly defined responsibilities.

Technology Flexibility

Individual services can use technologies appropriate to their responsibilities.

Easy Integration

APIs and events allow external systems to integrate with the platform.

Disadvantages

Increased Complexity

Managing many microservices is more complicated than managing a monolithic application.

Distributed Debugging

Tracing a problem across multiple services can be difficult.

Infrastructure Cost

The platform may require additional infrastructure for service management, messaging, monitoring, and deployment.

Data Consistency

Distributed services can make transaction management and consistency more challenging.

Operational Expertise

The development and operations teams require knowledge of distributed systems, cloud infrastructure, IoT, security, and monitoring.

6.5 Trade-Off Decision

Although the hybrid architecture introduces additional complexity, its benefits are more important for this project because the platform must support:

- Thousands of IoT devices
- Continuous telemetry
- Real-time processing
- Predictive analytics
- High availability
- Fault tolerance
- Multiple organizations
- Future expansion

Therefore, the additional architectural complexity is justified by the scalability and reliability requirements of the system.

6.6 Future Enhancements

The architecture is designed to support future features without major structural changes.

AI-Based Predictive Maintenance

AI models can predict refrigeration-unit failures.

Advanced Anomaly Detection

Machine-learning models can identify unusual combinations of environmental conditions.

Route Risk Prediction

The system can identify routes with high delivery or temperature-excursion risks.

Mobile Application

Drivers and field workers can receive alerts and shipment information through mobile devices.

Digital Twin

A digital representation of the shipment can be used for advanced simulation and risk analysis.

Automated Corrective Actions

Compatible refrigeration systems could potentially receive automated corrective instructions.

Multi-Cloud Deployment

The platform could be deployed across multiple cloud environments to improve resilience.

Advanced Regulatory Integration

The compliance service can integrate with external regulatory systems where required.

7. ARCHITECTURAL DECISION SUMMARY

Decision	Reason
Hybrid Architecture	Combines modular services with real-time event processing
Microservices	Independent development, deployment, and scaling
Event-Driven Communication	Suitable for continuous IoT telemetry
API Gateway	Secure and controlled service access
Message Broker	Handles high-volume asynchronous events
IoT Gateway	Connects sensors with cloud services
Separate Data Stores	Improves scalability and data organization
Analytics/AI Layer	Supports prediction and historical analysis
Role-Based Access	Protects sensitive operational information
Redundancy	Improves reliability and availability

8. CONCLUSION

The **Cold-Chain Logistics Integrity Platform** requires an architecture capable of processing continuous IoT telemetry while maintaining scalability, security, reliability, performance, availability, and maintainability.

A **Hybrid Microservices and Event-Driven Architecture** is therefore selected as the most suitable architecture. The microservices approach divides the platform into independent services such as telemetry, shipment management, threshold evaluation, alerting, GPS tracking, compliance, and analytics. The event-driven layer enables high-volume sensor events to be processed asynchronously and efficiently.

The proposed architecture provides a clear separation between IoT devices, communication infrastructure, processing services, data storage, analytics, presentation, and external integrations. This separation supports independent scaling, fault isolation, easier maintenance, and future expansion.

The architecture also addresses the major quality attributes required for the system. Scalability is achieved through independent service scaling, security through authentication and access

control, performance through asynchronous event processing, reliability through redundancy and persistent messaging, maintainability through modular services, and availability through fault-tolerant infrastructure.

Although the architecture introduces additional complexity compared with a monolithic system, the trade-off is justified because the platform must support thousands of connected devices, real-time monitoring, predictive analytics, multiple organizations, and continuous operation.

Overall, the proposed architecture provides a **scalable, secure, reliable, maintainable, and future-ready foundation** for intelligent cold-chain logistics monitoring across food and pharmaceutical distribution networks.